

Layered Architecture in Computer Networks



Layered architecture in computer networks refers to dividing a network's functioning into different layers, each responsible for a certain communication component.

The major goal of this layered architecture is to separate the complex network communication process into manageable, smaller activities that can be better developed.

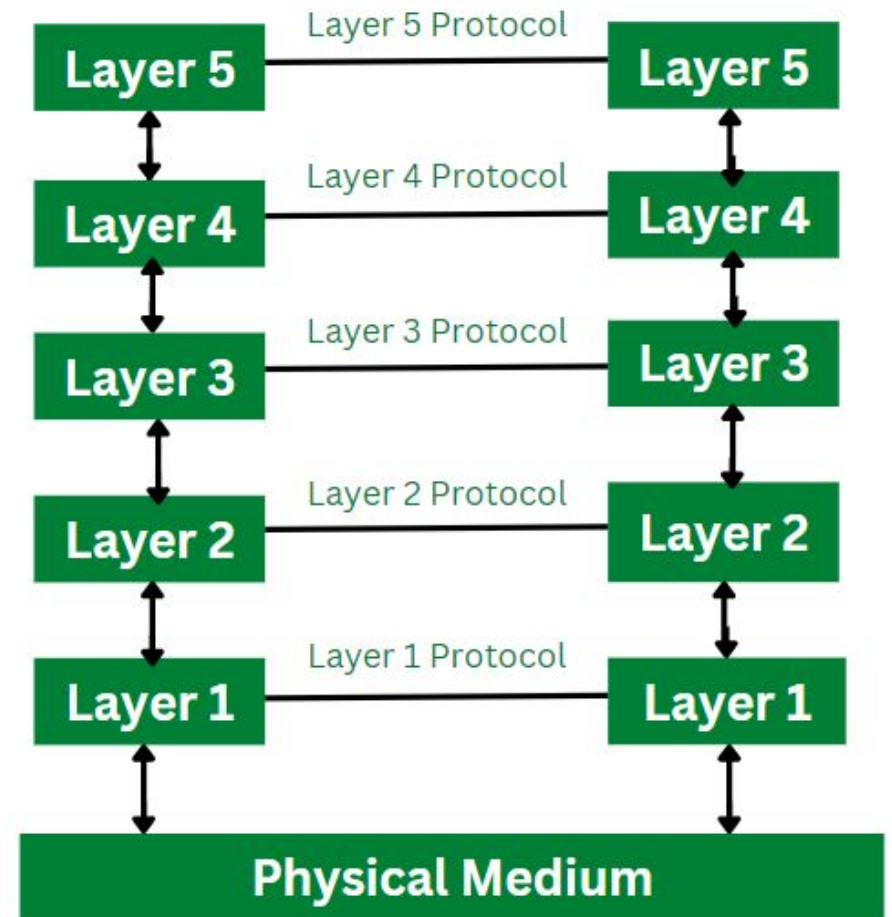
Every network consists of a specific number of functions, layers, and tasks to perform.

Layered Architecture in a computer network is defined as a model where a whole network process is divided into various smaller sub-tasks.

These divided sub-tasks are then assigned to a specific layer to perform only the dedicated tasks.

A single layer performs only a specific type of task. To run the application and provide all types of services to clients a lower layer adds its services to the higher layer present above it.

Therefore layered architecture provides interactions between the sub-systems. If any type of modification is done in one layer it does not affect the next layer.



Features of Layered Architecture

- Use of Layered architecture in computer network provides with the feature of modularity and distinct interfaces.
- Layered architecture ensures independence between layers, by offering services to higher layers from the lower layers and without specifying how these services are implemented.
- Layered architecture segments as larger and unmanageable design into small sub tasks.
- In layer architecture every network has different number of functions, layers and content.
- In layered architecture, the physical route provides with communication which is available under the layer 1.
- In layered architecture, the implementation done by one layer can be modified by another layer.

Elements of Layered Architecture

There are three different types of elements of a layered architecture.

1. **Service:** Service is defined as a set of functions and tasks being provided by a lower layer to a higher layer. Each layer performs a different type of task. Therefore, actions provided by each layer are different.
2. **Protocol:** Protocol is defined as a set rule used by the layer for exchanging and transmission of data with its peer entities. These rules can consists details regarding a type of content and their order passed from one layer to another.
3. **Interface:** Interface is defined as a channel that allows to transmit the messages from one layer to the another.

Significance of Layered Architecture

Divide and Conquer Approach: Layered architecture supports divide and conquer approach. The unmanageable and complex task is further divided into smaller sub tasks. Each sub task is then carried out by the different layer. Therefore, using this approach reduces the complexity of the problem or design process.

Easy to Modify: The layers are independent of each other in layered architecture. If any sudden change occurs in the implementation of one layer, it can be changed. This change does not affect the working of other layers involved in the task. Therefore, layered architectures are required to perform any sudden update or change.

Modularity: Layered architecture is more modular as compared to other architecture models in computer network. Modularity provides with more independence between the layers and are easier to understand.

Easy to Test: Each layer in layered architecture performs a different and dedicated task. Therefore, each layer can be analyzed and tested individually. It helps to analyze the problem and solve them more efficiently as compared to solving all the problems at a time.

Scalability: As networks grow in size and complexity, additional layers or protocols may be added to meet new requirements while maintaining existing functionality.

Security: The layered technique enables security measures to be implemented to varying degrees, protecting the community from a variety of threats.

Efficiency: Each layer focuses on a certain aspect of verbal exchange, optimizing aid allocation and performance.

1. OSI Model

OSI stands for Open Systems Interconnection. OSI is a seven layered architecture. All these seven layers work collaboratively to transmit data from one layer to another. Below are the layers of OSI Model.

Physical Layer: Physical layer is the lowest layer of OSI model and is responsible for the physical connection between all the required devices. The information present in physical layer is in the form of bits. Physical layer performs various functions such as bit rate control, bit synchronization, transmission mode etc.

Data Link Layer: Data Link layer provides with successful delivery of message from one node to the another. It checks whether this delivery of message is error free. Other functions performed by data link layer are error control, framing, flow control etc.

Network Layer: Network Layer is responsible for the transmission of data from one host to another host that is connected in different network. It performs other tasks such routing and logical addressing.

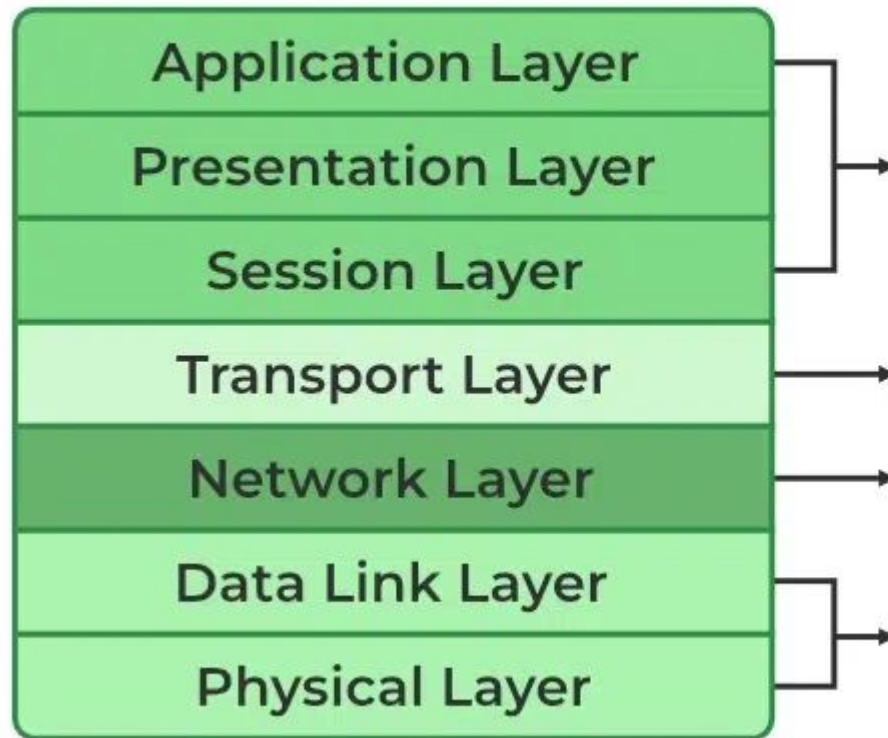
Transport Layer: Transport Layer is defined as a layer that takes services from network layer and provides services to application layer. Other tasks performed by transport layer are service point addressing, segmentation and reassembling.

Session Layer: Session layer is defined as a layer that is responsible for establishing a connection, maintenance of session and to provide with security. Other functions of session Layer are to establish session, termination and synchronization.

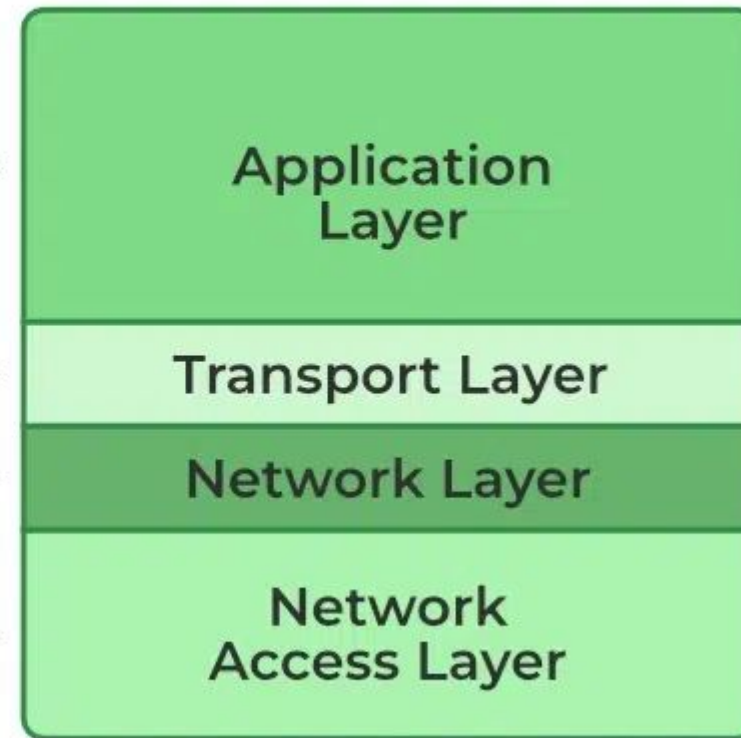
Presentation Layer: The data from application layer is extracted at the presentation layer. This layer is also known as translation layer. The functions of presentation layer are encryption, decryption, compression and translation.

Application Layer: Application layer is the topmost layer of OSI Model. Application layer is also known as desktop layer. It provides with other functions such as directory services, mail services, network virtual terminal etc.

OSI



TCP/IP



2. TCP/IP Model

1. Network Access Layer : It is a group of applications requiring network communications. This layer is responsible for generating the data and requesting connections. It acts on behalf of the sender and the Network Access layer on the behalf of the receiver. During this article, we will be talking on the behalf of the receiver. The packet's network protocol type, in this case, TCP/IP, is identified by network access layer. Error prevention and “framing” are also provided by this layer. Point-to-Point Protocol (PPP) framing and Ethernet IEEE 802.2 framing are two examples of data-link layer protocols.

2. Internet Layer This layer parallels the functions of OSI's Network layer. It defines the protocols which are responsible for the logical transmission of data over the entire network. The main protocols residing at this layer are as follows :

IP: IP stands for Internet Protocol and it is responsible for delivering packets from the source host to the destination host by looking at the IP addresses in the packet headers. IP has 2 versions: IPv4 and IPv6. IPv4 is the one that most websites are using currently. But IPv6 is growing as the number of IPv4 addresses is limited in number when compared to the number of users.

ICMP: ICMP stands for Internet Control Message Protocol. It is encapsulated within IP datagrams and is responsible for providing hosts with information about network problems.

ARP: ARP stands for Address Resolution Protocol. Its job is to find the hardware address of a host from a known IP address. ARP has several types: Reverse ARP, Proxy ARP, Gratuitous ARP, and Inverse ARP.

3. Transport Layer

The TCP/IP transport layer protocols exchange data receipt acknowledgments and retransmit missing packets to ensure that packets arrive in order and without error. End-to-end communication is referred to as such. Transmission Control Protocol (TCP) and User Datagram Protocol are transport layer protocols at this level (UDP).

TCP: Applications can interact with one another using TCP as though they were physically connected by a circuit. TCP transmits data in a way that resembles character-by-character transmission rather than separate packets. A starting point that establishes the connection, the whole transmission in byte order, and an ending point that closes the connection make up this transmission.

UDP: The datagram delivery service is provided by UDP, the other transport layer protocol. Connections between receiving and sending hosts are not verified by UDP. Applications that transport little amounts of data use UDP rather than TCP because it eliminates the processes of establishing and validating connections.

4. Application Layer

This layer is analogous to the transport layer of the OSI model. It is responsible for end-to-end communication and error-free delivery of data. It shields the upper-layer applications from the complexities of data.

HTTP and HTTPS: HTTP stands for Hypertext transfer protocol. It is used by the World Wide Web to manage communications between web browsers and servers. HTTPS stands for HTTP-Secure. It is a combination of HTTP with SSL(Secure Socket Layer). It is efficient in cases where the browser needs to fill out forms, sign in, authenticate, and carry out bank transactions.

SSH: SSH stands for Secure Shell. It is a terminal emulations software similar to Telnet. The reason SSH is preferred is because of its ability to maintain the encrypted connection. It sets up a secure session over a TCP/IP connection.

NTP: NTP stands for Network Time Protocol. It is used to synchronize the clocks on our computer to one standard time source. It is very useful in situations like bank transactions. Assume the following situation without the presence of NTP. Suppose you carry out a transaction, where your computer reads the time at 2:30 PM while the server records it at 2:28 PM. The server can crash very badly if it's out of sync.

Benefits of Layered Architecture

1. Modularity
2. Interoperability
3. Flexibility
4. Reusability
5. Scalability
6. Security

Challenges in Layered Architecture

1. Performance Overhead
2. Complexity in Implementation
3. Resource Utilization
4. Debugging and Troubleshooting
5. Protocol Overhead